Course Code: CSA1741 Course Name: ARTIFICIAL INTELLIGENCE
Slot: B
ASSIGNMENT 3 – CO3

Student Name: **Pavithra Sankar**
Niranjna devi S I
Reg. No: **192519100**
192312174

CO3: Logical agents – Propositional logic – Reasoning patterns in propositional logic – Resolution – First-order logic – Knowledge engineering in FOL – Inference in first-order logic – Propositional versus first-order inference – Unification and lifting – Forward chaining – Backward chaining – Resolution.

Mode: Pair Work – 2 Students per Group **Duration:** 1 Week **Total Marks: 50**
Submission Date: _____

Pair-Work Principle

Both students are responsible for understanding the complete assignment. Each task contains clearly identified individual responsibilities so that the contribution and understanding of each student can be independently verified.

Student A and Student B must not divide the assignment task-wise. Both students must participate in every task.The individual responsibilities alternate across tasks so that both students show evidence of:

Design → Implementation → Analysis → Evaluation.

TASK 1 – Logical Agents: Knowledge Bases, Entailment and Reasoning Patterns (6 Marks)

Part A – Knowledge Base and Model Checking in the Wumpus World

Shared Group Work: Together encode the rules of an $n \times n$ Wumpus world in propositional logic (pit, wumpus, breeze and stench symbols for every square and the rules relating percepts to neighbouring hazards). Implement a truth-table entailment checker and query it with percept sequences that you design, for example, “Is square [2,2] safe?”.

Student A – Design and Mathematical Analysis

- Count the proposition symbols and sentences as a function of n

- Derive the number of models 2^m and identify the minimal set of symbols relevant to one query
- Explain what soundness and completeness mean for model checking

Student B – Experimental Validation

- Instrument the checker to record models enumerated, KB evaluations and time
- Run it for n = 2, 3, 4 (state where the full symbol set becomes infeasible) and again with relevant-symbol restriction
- Include one entailed query, one query whose negation is entailed and one query for which neither is entailed

Part B – Reasoning Patterns

Shared Group Work: Using the same Wumpus-world KB, derive the safety of at least two squares by applying inference rules (Modus Ponens, And-Elimination, Unit Resolution) instead of model checking.

Student A – Experimental Investigation: Implement the inference-rule derivation, record the number of inference steps and compare with the model-checking cost for the same query.

Student B – Soundness Reasoning: Prove the soundness of each rule used with a truth table and explain why a derived sentence must hold in every model of the KB, and why "not derived" is not the same as "false".

Cross-Understanding Requirement: Each student must be able to explain the other student's component.

Evidence Expected: KB listing, symbol-count derivation, checker code and outputs for the three query types, inference-rule derivations with step counts, soundness tables and a comparison of the two reasoning approaches.

TASK 2 – The Quality of Reasoning Method: Model Checking versus Resolution (7 Marks)

Shared Group Work: Implement both a truth-table model checker and a propositional resolution prover (CNF conversion followed by resolution refutation). Design a smart-building access-control policy of at least 10 rules (badge validity, time of day, visitor status, emergency override) that contains implication, biconditional and disjunction.

The group must design test knowledge bases that make each method look strong or weak. Do not use a single sample KB.

Student A – Resolution Investigation

- Convert three sentences to CNF by hand and construct a resolution proof for one entailed query
- Record clauses generated, resolvents attempted, tautologies discarded, subsumed clauses removed and time as the KB grows (implication chains of length 10, 20, 40, 80)
- Test a query that is not entailed and explain why the process terminates

Student B – Model-Checking Comparative Analysis

- Record models enumerated and time for the same KBs and queries
- Identify a KB family on which model checking is faster than resolution and a family on which it is far slower, and explain what structural property causes the difference

Joint Design Question

If you were designing a reasoning component whose KB size and query types are not known in advance, what information would you want before deciding between these two methods? Support your answer with your own measurements.

Evidence Expected: Manual CNF conversion, proof tree, both implementations, clause and model counts across KB sizes, designed KB families and the joint design discussion.

TASK 3 – Breaking the Expressiveness Barrier: Knowledge Engineering in First-Order Logic (7 Marks)

Problem Context – University Exam-Eligibility Advisor

Regulations decide whether a student may write an examination (minimum attendance, prerequisites cleared, fees paid, disciplinary status, approved leave).

Shared Group Work: Follow the knowledge-engineering process (identify the task, assemble knowledge, decide vocabulary, encode general knowledge, encode the instance, pose queries, debug the KB). Build two encodings of the same regulations: a **propositional** one (one set of rules per student and course) and a **first-order** one (general axioms plus facts).

Student A – Design Derivation

- Define the FOL vocabulary and write at least 12 axioms with quantifiers, including one existential statement
- Derive how the number of sentences in the propositional encoding grows with students s and courses c, and show how the FOL axioms remain constant

Student B – Crossover Investigation

- Design at least 15 competency queries (5 that succeed, 5 that fail, 5 returning bindings) and document at least three KB defects found and fixed
- Measure KB size and inference time for both encodings as s and c grow, and locate the range of sizes at which the FOL encoding becomes preferable despite unification overhead

Joint Conclusion

Why does greater expressiveness not automatically give better reasoning performance for every KB size? Support your answer with your measurements.

Evidence Expected: Vocabulary and axioms, growth derivation, query suite with expected and actual results, defect log, size/time tables and a reasoned conclusion.

TASK 4 – The Reality of Unification and Lifted Inference (7 Marks)

The responsibilities in this task are reversed compared with Task 3.

Shared Group Work: Implement UNIFY (with substitution composition and an optional occurs check), lifted Modus Ponens, and propositionalisation of a small FOL KB (instantiating universal sentences with all ground terms).

Student B – Logical Design and Correctness

- Derive the most general unifier by hand for five pairs of terms that you design, including a pair where the occurs check matters and one where standardising apart is essential
- Explain why lifted inference is sound and why it can avoid generating ground instances

Student A – Experimental and Engineering Analysis

- Measure the number of ground clauses as the number of constants grows (k = 3 to 30) and as predicate arity grows

- Introduce a function symbol and show how the number of ground terms grows with the term-depth bound
- Compare inference steps and time for lifted versus propositional inference on the same query

Joint Engineering Decision

Under what conditions would you propositionalise a knowledge base despite the availability of lifted inference? State the conditions using your own evidence.

Evidence Expected: Unifier code and tests, hand derivations, ground-clause growth tables and plots, a lifted-versus-propositional comparison and the engineering decision.

TASK 5 – Upgrading Resolution from Propositional to First-Order Logic (6 Marks)

In Task 2, propositional resolution was applied to a policy KB. Review that solution before attempting this task.

Shared Group Work: Using the following English statements, design a first-order resolution prover:

1. Every employee who handled a restricted document is authorised.
2. Every document containing patient data is restricted.
3. Some employee handled every document that was audited.
4. Report R1 contains patient data and R1 was audited.

Convert the statements to FOL and CNF (with Skolemisation), implement resolution using your Task 4 unifier, and prove that an authorised employee exists.

Student A – Conversion and Proof Design: Show every conversion step, justify the Skolem function or constant, draw the proof tree and implement one resolution strategy of your choice (unit preference, set-of-support or input resolution).

Student B – Comparative Evidence: Record clauses generated, resolution steps, proof length and unifications with and without the strategy. Pose a second query that is **not** entailed and report the behaviour of your prover under a step cap. Compare with the propositional prover of Task 2 on an equivalent small domain.

Cross-Understanding Requirement

- Student B must explain why Skolemisation preserves satisfiability but not logical equivalence
- Student A must explain what the experimental evidence shows about the cost of lifting resolution to first-order logic, and what the non-entailed query shows about the decidability of first-order entailment

Evidence Expected: Conversion steps, proof tree, prover code, strategy comparison, behaviour on the non-entailed query and responses to the cross-understanding questions.

TASK 6 – Advanced Inference Mini-Investigations (7 Marks)

Both students investigate a different inference strategy on the same knowledge base (an organisation hierarchy with reports-to relations and approval rules, about 10 definite-clause rules, with 100, 1,000 and 10,000 facts).

Student A – Forward Chaining Investigation

- Implement forward chaining to a fixed point
- Record facts inferred, rule firings, redundant derivations, iterations to fixed point and the percentage of derived facts irrelevant to the query
- Explain what happens to the work when the query concerns only a very small part of the KB

Student B – Backward Chaining Investigation

- Implement backward chaining with substitutions
- Engineer a left-recursive rule on which naive backward chaining never terminates, implement a remedy (loop check, depth bound or memoisation) and evaluate its cost
- Record goals expanded, unifications, maximum depth and repeated subgoals

Verification Questions

- Student B must explain: *Why is the choice of inference direction central to performance on this KB?*
- Student A must explain: *Why can backward chaining remain efficient even when the KB contains many facts irrelevant to the query?*

Evidence Expected: Two working engines, metric tables for the three fact sizes, the non-termination demonstration, the remedy, and correct responses to the verification questions.

DAY-4 DESIGN CHALLENGE

TASK 7 – Design a Knowledge-Based Reasoning Component from First Principles (10 Marks)

Problem Context – Hospital Medication-Safety Advisor

Given a patient record (age, allergies, current medicines, conditions) and a newly prescribed drug, the agent must decide whether the prescription is safe and explain why. The initial knowledge base has about 40 rules. The hospital plans to grow it to 2,000 rules and 50,000 patients.

This problem must be solved collaboratively. Neither student may work independently and merge the answers later.

Stage 1 – Independent Design Proposal – 1 Mark

Before discussing with each other, Student A and Student B must each independently propose: the representation (propositional or FOL), vocabulary, inference method (forward chaining, backward chaining or resolution) and how explanations will be produced. Each initial proposal must be retained in the submission. No marks are deducted merely because an initial design is incorrect.

Stage 2 – Compare the Two Designs – 1 Mark

Identify where the designs agree and differ, their strengths and weaknesses, and which ideas will be retained.

Stage 3 – Joint Final Design – 3 Marks

Produce the final design: representation, vocabulary, inference method, indexing or agenda strategy for scale, explanation mechanism and complete pseudocode.

Stage 4 – Divide the Validation Responsibilities – 3 Marks

- **Student A – Theoretical Validation:** soundness and completeness of the chosen inference, complexity of a query as a function of KB size, and the effect of restricting the KB to definite clauses.

- **Student B – Experimental Validation:** at least 30 test cases including dangerous combinations, KB sizes of 40, 400 and 2,000 rules, metrics (query latency, inference steps, memory, explanation length) and a baseline comparison, with code and outputs as proof.

Stage 5 – Cross-Defence – 1 Mark

Student A explains the experimental evidence supporting the group's design. Student B explains why the theoretical complexity result applies to the group's chosen inference.

Stage 6 – Joint Evaluation – 1 Mark

Why is the problem suitable for a logical agent? What are the limits of the representation (uncertainty, incomplete data)? What should an AI designer check before choosing a knowledge-based approach?

CONTRIBUTION RECORD

Each pair must include this table at the end of the submission, reflecting the actual work completed.

Task	Student A – Primary Contribution	Student B – Primary Contribution	Joint Work
Task 1			
Task 2			
Task 3			
Task 4			
Task 5			
Task 6			
Task 7			

MARK DISTRIBUTION – ASSIGNMENT 3

Task	**Assessed Area**	**Marks**
Task 1	Logical agents, entailment and reasoning patterns	6
Task 2	Model checking versus resolution	7

Task	Assessed Area	Marks
Task 3	Knowledge engineering in FOL	7
Task 4	Unification and lifted inference	7
Task 5	First-order resolution	6
Task 6	Forward and backward chaining	7
Task 7	Day-4 Design Challenge	10
	TOTAL	**50**

Submission Requirements: one report per pair for Tasks 1–7 with the contribution record, source code for all implementations, raw test data, and the tables/graphs used. Operation counts (models, clauses, inference steps) are primary evidence; time is secondary.

INDIVIDUAL VERIFICATION AND VIVA MODEL

Primary contribution does not imply exclusive responsibility. A student must not answer "My partner did this section." For each student conduct a brief 4–5 minute verification:

1. **Primary responsibility** – one technical question from the component the student led.
2. **Partner's responsibility** – one question from a component primarily completed by the partner.
3. **Transfer question** – modify a condition and ask for the consequence, for example: "If you remove the occurs check, which of your test pairs now gives a wrong answer?" or "If the KB grows to 100,000 facts and queries are rare, would you change from forward to backward chaining?"

Overall Assessment Expectation: Represent → Infer → Implement → Measure → Debug → Evaluate → Defend.

Solutions

TASK 1 – Logical Agents: Knowledge Bases, Entailment and Reasoning Patterns

Part A – Knowledge Base and Model Checking in the Wumpus World

Question: Encode the rules of an n × n Wumpus world in propositional logic, implement a truth-table entailment checker, and query whether a square such as [2,2] is safe.

Answer:

Let Pij mean that square (i,j) contains a pit, Wij mean that it contains the Wumpus, Bij mean that a breeze is perceived, and Sij mean that a stench is perceived. Let Safeij mean that the square contains neither a pit nor the Wumpus.

For each square, we can define safety as: Safeij ↔ (¬Pij ∧ ¬Wij). A square has a breeze when at least one square next to it (up, down, left or right) contains a pit: Bij ↔ (P(i−1)j ∨ P(i+1)j ∨ Pi(j−1) ∨ Pi(j+1)), omitting neighbours outside the grid. In the same way, Sij ↔ (W(i−1)j ∨ W(i+1)j ∨ Wi(j−1) ∨ Wi(j+1)).

For a 2 × 2 example, the percept rules are:

B11 ↔ (P12 ∨ P21)

B12 ↔ (P11 ∨ P22)

B21 ↔ (P11 ∨ P22)

B22 ↔ (P12 ∨ P21)

S11 ↔ (W12 ∨ W21), with equivalent neighbour rules for the other squares.

We can take the starting square (1,1) as safe. This gives us the facts ¬P11 and ¬W11 are facts. If the agent observes ¬B11 and ¬S11, then neither adjacent square (1,2) nor (2,1) contains a pit or Wumpus. So, based on these observations, both neighbouring squares can be treated as safe in this example.

To answer a query about a square, we check whether the query is true in every possible model that satisfies the knowledge base. In symbols, KB ⊨ α if and only if there is no model in which KB is true and α is false.

Student A – Design and Mathematical Analysis

Question: Count proposition symbols and sentences as a function of n, derive 2^m models, identify relevant symbols, and explain soundness and completeness.

Answer:

There are n^2 squares. With four basic proposition symbols (P, W, B and S) per square, the total number of proposition symbols is $m = 4n^2$. If Safe is stored as an additional independent symbol, the count becomes $5n^2$; in this design Safe is derived and is not counted as an independent symbol.

Grid size n	Squares n^2	Symbols $m = 4n^2$	All truth assignments 2^m
2	4	16	65,536
3	9	36	68,719,476,736
4	16	64	18,446,744,073,709,551,616

The exact sentence count can change depending on how we expand each biconditional and handle squares along the edges of the grid. A compact rule set contains one breeze relation and one stench relation per square, plus the safety definition and initial facts. This means the number of rule groups grows roughly with n^2, but the number of possible truth assignments increases much faster, as $2^{(4n^2)}$.

For a query about square (2,2), only the pit and Wumpus symbols of that square and its immediate neighbours, together with the percept symbols connected to those neighbours, are needed. Symbols in distant, disconnected regions can be excluded if no rule links them to the query. Focusing only on these relevant symbols can make the search much smaller.

Soundness means that if the checker reports KB $\models \alpha$, then α is true in every model of KB. Completeness means that if α is true in every model of KB, the checker will report entailment. Exhaustive truth-table model checking is sound and complete for finite propositional knowledge bases, provided all assignments are evaluated correctly.

Student B – Experimental Validation

Question: Record models enumerated, KB evaluations and time for n = 2, 3, 4, use relevant-symbol restriction, and test three query types.

Answer:

The table below gives the number of possible assignments for each grid size. Full enumeration is practical for n = 2, but becomes impractical at n = 3 and clearly infeasible at n = 4 on an ordinary student laptop. A relevant-symbol query can be evaluated using only the local variables connected to that query.

n	Full symbols	Full assignments	What this means in practice
2	16	65,536	Can be enumerated in a short experiment.
3	36	68.7 billion	Full enumeration is generally impractical.
4	64	1.84 × 10^19	Infeasible by direct enumeration.

For reproducibility, record the actual elapsed time from the implementation rather than inserting an estimated time. A suitable measurement record is: n, number of variables, assignments visited, KB evaluations, query, result and elapsed seconds.

Query type	Example KB information	Query	Expected result
Entailed	¬P11, ¬W11, ¬B11, ¬S11 and neighbour rules	Safe12	True / entailed in the simplified 2×2 example
Negation entailed	P22 is a fact	¬P22	False; its negation is entailed
Neither entailed	No fact or rule determines P22	P22	Undetermined; neither P22 nor ¬P22 follows

A query is entailed when it is true in all KB models. Its negation is entailed when the query is false in all KB models. If some models make it true and other models make it false, neither side is entailed.

Part B – Reasoning Patterns

Question: Derive the safety of at least two squares using Modus Ponens, And-Elimination and Unit Resolution.

Answer:

Example derivation using a 2 × 2 world:

Given ¬B11 and B11 ↔ (P12 ∨ P21), the breeze rule gives ¬(P12 ∨ P21).

By De Morgan's law, ¬P12 ∧ ¬P21.

By And-Elimination, derive ¬P12 and separately derive ¬P21.

Given ¬S11 and S11 ↔ (W12 ∨ W21), derive ¬W12 and ¬W21.

Using Safeij ↔ (¬Pij ∧ ¬Wij), apply Modus Ponens to derive Safe12 and Safe21.

The two safe-square conclusions are Safe12 and Safe21. The number of inference steps should be counted from the exact implementation; in the above hand derivation, the key derived literals are ¬P12, ¬P21, ¬W12, ¬W21, Safe12 and Safe21.

Student A – Experimental Investigation

Question: Implement the inference-rule derivation, record inference steps and compare with model checking.

Answer:

A rule-based system keeps a list of known facts and applies a rule whenever all of its conditions are met. For the example above, the engine derives the four negative hazard literals and then the two safety conclusions. Here, the work mainly depends on how many rules can be applied. Model checking, on the other hand, may need to examine as many as 2^m assignments. For a small KB, both are fast; as unrelated symbols are added, the rule-based derivation can avoid exploring assignments that do not contribute to the query.

```
facts = {"not_B11", "not_S11"}
rules = [
({"not_B11"}, "not_P12"),
({"not_B11"}, "not_P21"),
({"not_S11"}, "not_W12"),
({"not_S11"}, "not_W21"),
({"not_P12", "not_W12"}, "Safe12"),
({"not_P21", "not_W21"}, "Safe21"),
]
steps = 0
changed = True
while changed:
changed = False
for premises, conclusion in rules:
if premises.issubset(facts) and conclusion not in facts:
facts.add(conclusion)
```

```
steps += 1
changed = True
print("Derived:", sorted(facts))
print("Inference steps:", steps)
```

Note: This compact demonstration represents the already-expanded logical consequences of the percept rules. A full submission implementation should also parse the original biconditionals and record the model-checker metrics.

Student B – Soundness Reasoning

Question: Prove the soundness of each rule using truth tables and explain why not-derived is not false.

Answer:

Modus Ponens: from P and (P → Q), infer Q. The only row in which both premises are true is the row P = True and Q = True; therefore the conclusion is true whenever both premises are true.

P	Q	P → Q	Both premises P and (P→Q)	Q
T	T	T	T	T
T	F	F	F	F
F	T	T	F	T
F	F	T	F	F

And-Elimination: from P ∧ Q, infer P; and from P ∧ Q, infer Q. Whenever the conjunction is true, both components are true.

P	Q	P ∧ Q	Conclusion P valid?	Conclusion Q valid?
T	T	T	Yes	Yes
T	F	F	Not applicable	Not applicable
F	T	F	Not applicable	Not applicable
F	F	F	Not applicable	Not applicable

Unit Resolution: from (P ∨ Q) and ¬P, infer Q. If ¬P is true, P is false. For the disjunction to remain true, Q must be true. Thus the rule is sound.

P	Q	P ∨ Q	¬P	Q
T	T	T	F	T

T	F	T	F	F
F	T	T	T	T
F	F	F	T	F

If the rule engine does not derive a sentence, that does not automatically mean the sentence is false. The engine may be incomplete, may not have the necessary rule, or may have stopped before reaching the relevant rule. We can conclude that it is false only if we derive its negation or find a valid model in which it is false.

TASK 2 – The Quality of Reasoning Method: Model Checking versus Resolution

Question: Implement truth-table model checking and propositional resolution for a smart-building access-control policy containing at least 10 rules, including implication, biconditional and disjunction.

Answer:

Let V = valid badge, T = within permitted time, E = employee, G = registered guest, H = host approval, M = security manager approval, X = emergency override, A = access granted, D = door unlocked, and L = access logged.

Rule	Policy rule
R1	$E \rightarrow V$
R2	$G \rightarrow V$
R3	$V \rightarrow (A \vee X)$
R4	$T \rightarrow A$
R5	$G \rightarrow H$
R6	$H \rightarrow A$
R7	$X \rightarrow M$
R8	$M \rightarrow A$
R9	$A \leftrightarrow D$
R10	$A \rightarrow L$
R11	$\neg V \rightarrow \neg A$
R12	$(E \vee G) \rightarrow L$

The policy includes implication (→), biconditional (↔) and disjunction (∨). These rules are used only as an example for the assignment; they should not be treated as an actual building security policy. The knowledge base can be tested with different fact sets: a valid employee

during permitted time; a guest without host approval; an emergency override without manager approval; and a person with an invalid badge.

Student A – Resolution Investigation

Question: Convert three sentences to CNF by hand, construct a resolution proof for one entailed query, measure clause statistics for chain lengths 10, 20, 40 and 80, and test a non-entailed query.

Answer:

CNF conversions:

$V \rightarrow (A \vee X) \equiv \neg V \vee A \vee X.$

$A \leftrightarrow D \equiv (\neg A \vee D) \wedge (\neg D \vee A).$

$(E \vee G) \rightarrow L \equiv \neg(E \vee G) \vee L \equiv (\neg E \wedge \neg G) \vee L \equiv (\neg E \vee L) \wedge (\neg G \vee L).$

Resolution proof: Given T and rule $T \rightarrow A$, the CNF clauses are $\{T\}$ and $\{\neg T, A\}$. Resolving them on T gives $\{A\}$. Therefore $KB \models A$.

For a non-entailed query such as A when the KB contains only $\neg V$ and no emergency or independent access fact, the prover cannot derive A. The resolution process eventually stops when it cannot produce any new useful clause. If the process does not produce the empty clause, we have not proved the query. For a finite propositional clause set, this process terminates because only finitely many distinct clauses can be formed.

Implication-chain length	Initial chain clauses	What we expect to see
10	Approximately 10–11	Small clause set; quick saturation.
20	Approximately 20–21	More possible resolvent pairs.
40	Approximately 40–41	Pair comparisons grow rapidly without indexing.
80	Approximately 80–81	Naive resolution can spend substantial time on irrelevant pairs.

Record actual clauses generated, attempted resolvents, discarded tautologies, removed subsumed clauses and elapsed time from the running prover. The table above gives the chain setup, not measured runtime results.

Student B – Model-Checking Comparative Analysis

Question: Record models and time, identify a family where model checking is faster and one where it is slower, and explain why.

Answer:

Model checking enumerates all assignments to the symbols in the KB. With m symbols, it may inspect 2^m assignments. It is attractive when m is small, the KB is compact, and the query can be evaluated directly. Resolution is attractive when the clauses are sparse and a short proof can be found without exploring most assignments.

KB family	Method likely to work better	Reason
Small KB with 6–10 symbols	Model checking	At most 1,024 assignments for 10 symbols; simple enumeration has low overhead.
Long sparse implication chain	Resolution	A query may follow a short chain of relevant clauses without enumerating every truth assignment.
Dense clauses with many complementary pairs	Depends on implementation	Resolution may generate many resolvents; model checking may be competitive for a small variable set.

For the required measurements, use the same KB and query for both methods. Report model assignments visited, clauses generated, resolvent attempts and elapsed time. For a fair comparison, both methods should be run on the same computer, using the same language and stopping conditions.

Joint Design Question

Question: What information is needed before choosing between model checking and resolution?

Answer:

Before choosing a method, I would look at how many symbols and clauses the knowledge base contains, how closely connected those clauses are, how often queries are asked, and whether the knowledge base changes often. I would also check whether we need a short proof or an

explanation for each result. I would also measure the average number of models visited and resolution operations on representative KB families. For a small fixed vocabulary, model checking is simple and predictable. For sparse rule-based KBs with useful unit clauses, resolution may be more efficient. The choice should be based on measurements rather than KB size alone.

TASK 3 – Breaking the Expressiveness Barrier: Knowledge Engineering in First-Order Logic

Question: Build propositional and FOL encodings of university exam-eligibility regulations using the knowledge-engineering process.

Answer:

Task: decide whether a student is eligible to write a course examination. Knowledge sources: attendance rules, prerequisite completion, fee payment, disciplinary status and approved leave. Vocabulary: Student(x), Course(c), Attendance(x,c,a), RequiredAttendance(c,r), ClearedPrerequisite(x,c), FeesPaid(x,c), DisciplinaryHold(x), ApprovedLeave(x,c), Eligible(x,c).

In propositional logic, we would create separate symbols and rules for each student and course combination. With FOL, we can write the general rules once and store individual students and courses as facts. Example FOL rule: $\forall x \forall c[(Student(x) \land Course(c) \land AttendanceOK(x,c) \land PrerequisiteOK(x,c) \land FeesPaid(x,c) \land \neg DisciplinaryHold(x) \land (ApprovedLeave(x,c) \lor AttendanceThresholdMet(x,c))) \rightarrow Eligible(x,c)]$.

Student A – Design Derivation

Question: Define FOL vocabulary and at least 12 quantified axioms, including an existential statement, and derive growth of the two encodings.

Answer:

Predicates: Student(x), Course(c), AttendancePercent(x,c,a), AttendanceOK(x,c), PrerequisiteCleared(x,c), FeesPaid(x,c), DisciplinaryHold(x), ApprovedLeave(x,c), AttendanceRequirement(c,r), MeetsAttendance(x,c), Eligible(x,c), Ineligible(x,c), HasExam(x,c), and AdvisorAssigned(x).

Axioms (illustrative regulations):

A1. $\forall x \forall c[(Student(x) \land Course(c) \land AttendanceOK(x,c) \land PrerequisiteCleared(x,c) \land FeesPaid(x,c) \land \neg DisciplinaryHold(x)) \rightarrow Eligible(x,c)]$.

A2. ∀x∀c[(Student(x) ∧ Course(c) ∧ ApprovedLeave(x,c) ∧ PrerequisiteCleared(x,c) ∧ FeesPaid(x,c) ∧ ¬DisciplinaryHold(x)) → Eligible(x,c)].

A3. ∀x∀c[Eligible(x,c) → Student(x)].

A4. ∀x∀c[Eligible(x,c) → Course(c)].

A5. ∀x∀c[(Student(x) ∧ Course(c) ∧ DisciplinaryHold(x)) → Ineligible(x,c)].

A6. ∀x∀c[(Student(x) ∧ Course(c) ∧ ¬FeesPaid(x,c)) → Ineligible(x,c)].

A7. ∀x∀c[(Student(x) ∧ Course(c) ∧ ¬PrerequisiteCleared(x,c)) → Ineligible(x,c)].

A8. ∀x∀c∀a∀r[(AttendancePercent(x,c,a) ∧ AttendanceRequirement(c,r) ∧ a ≥ r) → AttendanceOK(x,c)].

A9. ∀x∀c[AttendanceOK(x,c) → MeetsAttendance(x,c)].

A10. ∀x∀c[ApprovedLeave(x,c) → AttendanceException(x,c)].

A11. ∀x∀c[(Eligible(x,c) ∧ HasExam(x,c)) → MayWriteExam(x,c)].

A12. ∀x[AdvisorAssigned(x) → Student(x)].

A13. ∃x[Student(x) ∧ AdvisorAssigned(x)].

The existential axiom states that at least one student has an assigned advisor. It introduces an existence claim rather than naming a specific student.

For s students and c courses, a propositional design may need O(sc) instance facts and rule groups because each student-course pair has its own symbols. The number of general FOL rules stays almost the same as the database grows, although more facts are needed for additional students and courses. FOL therefore reduces repeated rule descriptions, although it requires unification and variable substitution during inference.

Student B – Crossover Investigation

Question: Design 15 competency queries (5 succeed, 5 fail, 5 return bindings), document three defects, and compare size/time as s and c grow.

Answer:

Example facts for testing: Student(Anu), Student(Bala), Course(BME101), Course(AI101), AttendancePercent(Anu,BME101,90), AttendanceRequirement(BME101,75), PrerequisiteCleared(Anu,BME101), FeesPaid(Anu,BME101), ¬DisciplinaryHold(Anu), and HasExam(Anu,BME101). Additional facts can be added for failing cases.

Category	Five example queries	Expected result
Success	Eligible(Anu,BME101); AttendanceOK(Anu,BME101);	True

	MeetsAttendance(Anu,BME101); Student(Anu); Course(BME101)	
Failure	Eligible(Bala,AI101); FeesPaid(Bala,AI101); PrerequisiteCleared(Bala,AI101); ApprovedLeave(Bala,AI101); MayWriteExam(Bala,AI101)	False or not provable from the stated facts
Bindings	Eligible(x,BME101); AttendancePercent(Anu,c,a); AttendanceRequirement(c,r); Student(x); HasExam(x,c)	Return substitutions such as x=Anu, c=BME101, a=90, r=75 where supported

Three example KB defects and fixes:

Defect 1: The attendance rule used a > comparison, excluding students exactly at the required percentage. Fix: use a ≥ r comparison.

Defect 2: Approved leave was recorded but not connected to the eligibility rule. Fix: add the approved-leave eligibility alternative.

Defect 3: A disciplinary hold was missing from the eligibility condition. Fix: explicitly require ¬DisciplinaryHold(x).

Illustrative growth experiment plan:

Students (s)	Courses (c)	Pairs (s×c)	Propositional instance groups	FOL general axioms
10	5	50	Proportional to 50	12–13
100	20	2,000	Proportional to 2,000	12–13
500	50	25,000	Proportional to 25,000	12–13

Measure file size or stored sentence count and inference time using identical queries. The FOL rule count remains constant, but inference time can rise because the engine must unify variables with a larger set of facts. The point at which one method becomes faster depends on the implementation. We should find it by running the tests rather than guessing a value for the report.

Joint Conclusion

Question: Why does greater expressiveness not automatically give better reasoning performance for every KB size?

Answer:

FOL is more expressive because a single rule with variables can apply to many students and courses. However, the system still has to match those variables with facts, apply substitutions and check the possible matches. For a very small domain, a propositional KB may be faster because all relevant instances are already explicit. As the domain grows, FOL can save storage and rule duplication. Therefore, expressiveness improves representation and scalability of knowledge description, but does not guarantee lower inference time.

TASK 4 – The Reality of Unification and Lifted Inference

Question: Implement UNIFY, lifted Modus Ponens and propositionalisation; derive five most general unifiers (MGUs), including occurs-check and standardising-apart cases.

Answer:

A substitution maps variables to terms. A most general unifier (MGU) is a substitution that makes two expressions match while keeping the substitution as general as possible. The occurs check makes sure that a variable is not replaced with a term that already contains that variable.

Pair	MGU / result	Explanation
Knows(x, Ravi) and Knows(Anu, y)	{x/Anu, y/Ravi}	Both argument positions are unified.
Parent(x, y) and Parent(Anu, Bala)	{x/Anu, y/Bala}	Variables are replaced by matching constants.
Treat(x, x) and Treat(Anu, Anu)	{x/Anu}	Repeated variable must map consistently.
P(x) and P(f(x))	Failure with occurs check	x cannot be replaced by f(x), because x occurs inside f(x).
R(x, y) and R(y, x) after standardising apart	{x1/y2, y1/x2} (one valid renaming-based MGU)	Rename variables in one clause first so unrelated

		clause variables are not accidentally identified.

Before unifying two clauses, we standardise them apart by giving their variables different names. This keeps the variables in the two clauses independent. For example, rename Parent(x,y) from one rule to Parent(x1,y1) and the other clause variables to x2,y2. This prevents accidental variable capture.

Student B – Logical Design and Correctness

Question: Explain why lifted inference is sound and how it avoids generating all ground instances.

Answer:

Lifted inference works with variables directly. It uses unification to find the substitutions needed to answer a query. It is sound because the unifier makes the premises and conclusion consistent under a valid substitution, and the underlying inference rule preserves truth. This means we do not have to create every possible ground instance before starting the inference. Only substitutions relevant to the current inference are produced, reducing unnecessary work when the domain is large.

Student A – Experimental and Engineering Analysis

Question: Measure ground clauses for k = 3 to 30 and increasing predicate arity; introduce a function symbol and compare lifted and propositional inference.

Answer:

For a predicate of arity r over k constants, the number of possible ground atoms is k^r. If all tuples are allowed, the growth is:

Constants k	Unary (r=1)	Binary (r=2)	Ternary (r=3)
3	3	9	27
5	5	25	125
10	10	100	1,000
20	20	400	8,000
30	30	900	27,000

With a function symbol f of arity 1 and a term-depth bound d, the number of terms can grow rapidly. With k constants and one unary function, the terms up to depth d are approximately k(d+1) if only the chain f(c), f(f(c)), etc. is generated. With multiple function symbols or higher

arity functions, the number can grow exponentially with depth. If we do not set a limit on term depth, the Herbrand universe can become infinite.

Lifted inference can avoid creating all k^r ground atoms if only a small number of substitutions match the query. Propositionalisation may be preferable when the domain is tiny and finite, all ground instances are reused many times, or a simple SAT/model-checking engine is already available. Actual inference steps and runtime must be measured on the same query and same KB.

Joint Engineering Decision

Question: When would propositionalisation be selected despite lifted inference?

Answer:

I would propositionalise when the domain is small, fixed and finite; the number of ground instances is manageable; and the same ground clauses are queried repeatedly. I would prefer lifted inference when the domain is larger, many possible groundings are irrelevant, or the KB contains reusable general rules. We should make this choice after comparing the number of ground clauses, successful unifications, inference steps and actual running time.

TASK 5 – Upgrading Resolution from Propositional to First-Order Logic

Question: Convert the four English statements to FOL and CNF, implement first-order resolution using the Task 4 unifier, and prove that an authorised employee exists.

Answer:

Predicates: Employee(x), Handled(x,d), Authorised(x), PatientData(d), Restricted(d), Audited(d), and Contains(R1, patient_data).

FOL translations:

$\forall x \forall d[(Employee(x) \land Handled(x,d) \land Restricted(d)) \rightarrow Authorised(x)]$.

$\forall d[PatientData(d) \rightarrow Restricted(d)]$.

$\exists x[Employee(x) \land \forall d(Audited(d) \rightarrow Handled(x,d))]$.

$PatientData(R1) \land Audited(R1)$.

CNF conversion:

Statement 1: $\neg Employee(x) \lor \neg Handled(x,d) \lor \neg Restricted(d) \lor Authorised(x)$.

Statement 2: $\neg PatientData(d) \lor Restricted(d)$.

Statement 3: Skolemise the existential employee using a Skolem constant e: Employee(e) and $\neg Audited(d) \lor Handled(e,d)$.

Statement 4: PatientData(R1) and Audited(R1).

Here, the Skolem constant e stands for an employee whose existence is stated in the third sentence. It is not the name of an actual employee; it is simply a new constant used in the logic.

Student A – Conversion and Proof Design

Question: Show conversion steps, justify Skolemisation, draw the proof tree and select a resolution strategy.

Answer:

Resolution proof:

From PatientData(R1) and ¬PatientData(R1) ∨ Restricted(R1), resolve to obtain Restricted(R1).

From Audited(R1) and ¬Audited(R1) ∨ Handled(e,R1), resolve to obtain Handled(e,R1).

We already have Employee(e).

Instantiate the first universal clause with x=e and d=R1: ¬Employee(e) ∨ ¬Handled(e,R1) ∨ ¬Restricted(R1) ∨ Authorised(e).

Resolve successively with Employee(e), Handled(e,R1) and Restricted(R1) to obtain Authorised(e).

So we can conclude that at least one authorised employee exists, and e is the witness for that conclusion. A set-of-support strategy is suitable because the negated query is placed in the support set and resolution focuses on clauses connected to the query.

Student B – Comparative Evidence

Question: Record clauses, resolution steps, proof length and unifications with/without strategy; test a non-entailed query under a step cap; compare with propositional resolution.

Answer:

For the proof above, the initial CNF contains six clause groups if the two facts in statement 4 are counted separately and the two Skolemised clauses are counted separately. The proof requires three principal intermediate derivations: Restricted(R1), Handled(e,R1), and Authorised(e). Exact unification counts depend on whether the implementation counts constant matches and variable substitutions separately.

A non-entailed query can be “Authorised(Anu)” when no fact identifies e with Anu and no rule establishes Anu’s authorisation. If the step limit is reached, the prover should say “not proved within the limit” instead of saying that the query is false. First-order resolution can search an

unbounded space when function symbols are present, so failure to find a proof within a finite cap does not establish non-entailment.

On an equivalent small finite domain, propositional resolution can ground the employee and document constants explicitly. It may be simpler and faster for a tiny domain, while FOL resolution avoids manually creating repeated ground clauses as the domain expands.

Cross-Understanding Requirement

Question: Explain why Skolemisation preserves satisfiability but not logical equivalence, and what the experiment shows about first-order resolution.

Answer:

During Skolemisation, an existential variable is replaced with a new Skolem constant or function. The new formula is equisatisfiable with the original one: the original formula has a model if and only if the Skolemised formula has a corresponding model in an expanded vocabulary. However, the formulas are not logically equivalent in the original vocabulary because the Skolem symbol is new and the transformed formula makes a particular witness explicit.

The experiment shows that first-order resolution has additional cost from unification, substitutions and potentially many ground terms. The non-entailed query demonstrates that a prover may not terminate on every first-order input, especially with unrestricted function symbols. A step cap is useful operationally, but a capped search result must be reported as inconclusive.

TASK 6 – Advanced Inference Mini-Investigations

Question: Implement forward chaining and backward chaining over an organisation hierarchy with about 10 definite-clause rules and 100, 1,000 and 10,000 facts.

Answer:

Example predicates: ReportsTo(employee, manager), Manager(x), Approves(x, request), CanApprove(x, request), and SeniorManager(x). Example rules:

ReportsTo(x,y) ∧ Manager(y) → Manager(x).

Manager(x) ∧ Approves(x,r) → CanApprove(x,r).

ReportsTo(x,y) ∧ CanApprove(y,r) → CanApprove(x,r).

SeniorManager(x) → Manager(x).

Manager(x) ∧ ReportsTo(y,x) → Supervises(x,y).

Supervises(x,y) ∧ CanApprove(x,r) → CanApprove(y,r).

Manager(x) ∧ HasTraining(x) → TrainedManager(x).

TrainedManager(x) ∧ Approves(x,r) → ValidApproval(x,r).

ReportsTo(x,y) ∧ ReportsTo(y,z) → ReportsToIndirectly(x,z).

ValidApproval(x,r) → RequestApproved(r).

Student A – Forward Chaining Investigation

Question: Implement forward chaining to a fixed point and record inferred facts, rule firings, redundant derivations, iterations and irrelevant derived facts.

Answer:

Forward chaining is a data-driven method. It starts with the facts we already know, applies rules whose conditions are satisfied, adds the new conclusions and continues until nothing new can be inferred. A simple fixed-point engine is:

```
def forward_chain(facts, rules):
known = set(facts)
firings = 0
iterations = 0
changed = True
while changed:
changed = False
iterations += 1
for premises, conclusion in rules:
if premises.issubset(known):
firings += 1
if conclusion not in known:
known.add(conclusion)
changed = True
return known, firings, iterations
```

Forward chaining may derive many facts that are unrelated to a particular query because it attempts to reach a fixed point for the whole rule set. As the KB grows from 100 to 10,000 facts, the number of possible rule matches and redundant attempts can increase. Indexing facts by predicate and argument can reduce unnecessary scans.

Fact count	What to observe during testing	Likely issue

100	Small number of rule firings and iterations	Low overhead
1,000	More matches and derived facts	Repeated scans become visible
10,000	Potentially many irrelevant derived facts	Need indexing and duplicate suppression

Student B – Backward Chaining Investigation

Question: Implement backward chaining with substitutions, demonstrate a left-recursive non-terminating rule, apply a remedy and record goals/unifications/depth.

Answer:

Backward chaining works in the opposite direction. It starts with the question we want to answer and looks for rules that could prove it. The rule premises become subgoals. It can avoid processing large sections of the KB that are irrelevant to the requested goal.

A left-recursive rule such as Ancestor(x,z) :- Ancestor(x,y), Parent(y,z) can cause naive depth-first backward chaining to expand Ancestor repeatedly before reaching a base fact. One way to avoid this is to keep track of goals that have already appeared in the current search path. Memoisation or a maximum depth limit can also help. A depth bound prevents infinite recursion but may miss valid proofs deeper than the limit.

```
def backward_chain(goal, facts, rules, path=None, depth=0, max_depth=50):
if path is None:
path = set()
if depth > max_depth or goal in path:
return False
if goal in facts:
return True
path = path | {goal}
for head, body in rules:
# In a full FOL implementation, unify head with goal
# and apply the resulting substitution to every body goal.
if head == goal:
if all(backward_chain(g, facts, rules, path, depth+1, max_depth)
for g in body):
```

return True

return False

The code is a propositional-shaped control-flow illustration; a complete FOL version must call the Task 4 UNIFY function and apply substitutions to the head and body. Record goals expanded, unifications, maximum recursion depth and repeated subgoals.

Verification Questions

Question: Why is inference direction central to performance, and why can backward chaining remain efficient with many irrelevant facts?

Answer:

The direction of inference decides whether the system works through the consequences of all known facts or focuses only on the rules needed for a particular query. Forward chaining is useful when many conclusions are needed or new facts arrive continuously. Backward chaining is useful when queries are specific and only a small part of the KB is relevant. With indexing by predicate and goal arguments, backward chaining can ignore unrelated facts and rules, even when the total fact count is large.

DAY-4 DESIGN CHALLENGE – TASK 7: Design a Knowledge-Based Reasoning Component from First Principles

Question: Design a hospital medication-safety advisor for patient records and prescribed drugs, scaling from 40 to 2,000 rules and 50,000 patients.

Answer:

The advisor takes details such as the patient's age, allergies, current medicines, diagnosed conditions and the newly prescribed medicine as input. It then gives a safety result along with an explanation that a person can understand. It is a decision-support system and must not replace the clinical judgement of a qualified healthcare professional.

Stage 1 – Independent Design Proposal

Student A – Initial proposal:

Use first-order logic because rules must apply to many patients and medicines. Vocabulary includes Patient(p), Drug(d), HasAllergy(p,a), ContainsIngredient(d,a), Takes(p,d), Contraindicated(d,c), HasCondition(p,c), and Unsafe(p,d). Use backward chaining for a specific patient-drug query. Explanations are generated by storing the rule and facts used in each proof.

Student B – Initial proposal:

Use a rule-based FOL knowledge base with a combination of forward chaining for patient-record updates and backward chaining for prescription checks. Use an indexed rule agenda and a trace of matched rules to produce explanations. Add an explicit "insufficient information" result when required patient data is missing.

These are draft independent proposals. The pair should retain the actual separately prepared versions and edit them only after discussion.

Stage 2 – Compare the Two Designs

Question: Identify agreement, differences, strengths and retained ideas.

Answer:

Aspect	Student A	Student B	Decision we agreed on
Representation	FOL	FOL	Use FOL for reusable patient/drug rules.
Inference	Backward chaining	Forward + backward chaining	Backward chaining for prescription queries; forward updates only when needed.
Scale strategy	General rules	Indexed agenda	Index by predicate, drug, ingredient and patient.
Explanation	Proof trace	Rule trace plus missing-data state	Store facts and rules supporting each result.

Stage 3 – Joint Final Design

Question: Give representation, vocabulary, inference method, indexing/agenda, explanation mechanism and complete pseudocode.

Answer:

Final representation: First-order definite clauses for deterministic rules, with explicit three-valued outcome handling: SAFE UNDER CHECKED RULES, UNSAFE, or INSUFFICIENT DATA. The system should not treat missing information as evidence of safety.

Vocabulary: Patient(p), Drug(d), Age(p,a), HasAllergy(p,i), Ingredient(d,i), Takes(p,d), Interaction(d1,d2), HasCondition(p,c), ContraindicatedFor(d,c), Dose(p,d,q), MaxDose(d,qmax), and Unsafe(p,d,reason).

Example rules:

HasAllergy(p,i) ∧ Ingredient(d,i) → Unsafe(p,d,allergy_match).

Takes(p,d1) ∧ Interaction(d,d1) → Unsafe(p,d,drug_interaction).

HasCondition(p,c) ∧ ContraindicatedFor(d,c) → Unsafe(p,d,condition_conflict).

Dose(p,d,q) ∧ MaxDose(d,qmax) ∧ q > qmax → Unsafe(p,d,dose_limit).

If all required checks have complete data and no unsafe rule fires, return SAFE UNDER CHECKED RULES.

Indexing strategy: index allergy rules by ingredient, interaction rules by drug pair, contraindication rules by condition and drug, and dose rules by drug. Use an agenda so only rules affected by a changed patient fact are reconsidered.

Explanation mechanism: every conclusion stores the rule ID, matched patient facts, matched drug facts and substitution. The explanation generator converts this proof trace into a sentence such as “Flagged because the patient record lists an allergy to ingredient X, and the prescribed medicine contains ingredient X.”

Pseudocode:

```
function checkPrescription(patient, prescribedDrug):
required = [allergy_data, current_medicines, conditions,
age, dose_information]
missing = findMissingRequiredData(patient, prescribedDrug, required)
if missing is not empty:
return INSUFFICIENT_DATA with explanation(missing)
candidateRules = retrieveIndexedRules(prescribedDrug, patient)
agenda = candidateRules
fired = set()
explanations = []
while agenda is not empty:
rule = agenda. Pop ()
match = unify (rule. premises, patient, prescribedDrug)
if match succeeds and (rule.id, match) not in fired:
```

fired. Add ((rule.id, match))

conclusion = apply (rule, match)

if conclusion is Unsafe:

explanations.append(rule.id, match, conclusion.reason)

if explanations is not empty:

return UNSAFE with explanations

else:

return SAFE_UNDER_CHECKED_RULES with checkedRuleList

A qualified clinician must review the result before any final decision is made. The system should record the knowledge-base version and timestamp so that each explanation can be audited.

Stage 4 – Divide the Validation Responsibilities

Question: Student A validates soundness/completeness and complexity; Student B tests at least 30 cases at 40, 400 and 2,000 rules with metrics and a baseline.

Answer:

Student A – Theoretical validation: For a definite-clause rule base, forward chaining is sound under standard Horn-clause semantics. With fair rule application and a finite set of ground facts, it reaches the least fixed point and is complete for ground atomic queries. A naive implementation may repeatedly scan all rules and facts; indexing and semi-naive evaluation reduce repeated work. Restricting the KB to definite clauses makes inference more manageable, but cannot directly represent every form of uncertainty or arbitrary non-Horn knowledge.

Student B – Experimental validation: Use 30 test cases covering allergy matches, drug interactions, contraindications, dose limits, safe cases, missing records and conflicting facts. Repeat the same cases at 40, 400 and 2,000 rules. Compare the proposed indexed system against a baseline that scans all rules.

Test group	Cases	Expected outcome
Allergy match / no match	1–5	Unsafe for matching ingredient; no allergy flag otherwise

Drug interaction	6–10	Unsafe when a known interacting medicine is present
Condition contraindication	11–15	Unsafe when the patient has a contraindicated condition
Dose limit	16–20	Unsafe when prescribed dose exceeds the stored limit
Missing or incomplete data	21–25	Insufficient data; do not assume safe
Safe and boundary cases	26–30	Safe under checked rules only when all required checks pass

Record query latency (milliseconds), inference steps, peak memory and explanation length. Use the same hardware and dataset for the baseline and proposed system. The table below is the measurement template; fill it with actual results:

Rules	Method
40	Full-scan baseline
40	Indexed design
400	Full-scan baseline
400	Indexed design
2,000	Full-scan baseline
2,000	Indexed design

Do not fill this table with invented benchmark numbers. Run the same 30 cases against both versions and insert the measured values.

Stage 5 – Cross-Defence

Question: Student A explains experimental evidence; Student B explains theoretical complexity.

Answer:

Student A should explain whether indexing reduced the number of rules examined, whether latency increased as the KB grew, and whether the explanation trace remained understandable. Student B should explain that query cost depends on the number of relevant rules and matching facts, not only the total number of rules. In a naive implementation, repeated scans can approach a large product of rule count and fact count; indexing reduces the candidate set but does not remove the cost of unification or large numbers of matching facts.

Stage 6 – Joint Evaluation

Question: Why is this problem suitable for a logical agent? What are the limits of the representation, and what should an AI designer check?

Answer:

Medication-safety checking is suitable for a logical agent because many decisions are based on explicit relationships and rules: allergies, ingredients, interactions, contraindications and dose limits. A logical agent can provide traceable explanations showing which facts and rules caused a warning.

One important limitation is that patient information may be incomplete or uncertain. Patient records may be missing, outdated or inconsistent. Clinical risk may depend on context, probability, laboratory values and specialist judgement that simple Boolean rules cannot fully represent. Before choosing a knowledge-based approach, the designer should check whether reliable rules exist, whether the data are complete and current, how conflicts are resolved, how explanations are audited, how the system is validated, and how clinicians remain responsible for final decisions.

TASK 1 – Logical Agents: Knowledge Bases, Entailment and Reasoning Patterns

Part A – Knowledge Base and Model Checking in the Wumpus World

Question: Encode the rules of an n × n Wumpus world in propositional logic, implement a truth-table entailment checker, and query whether a square such as [2,2] is safe.

Answer:

Let Pij mean that square (i,j) contains a pit, Wij mean that it contains the Wumpus, Bij mean that a breeze is perceived, and Sij mean that a stench is perceived. Let Safeij mean that the square contains neither a pit nor the Wumpus.

For each square, we can define safety as: Safeij ↔ (¬Pij ∧ ¬Wij). A square has a breeze when at least one square next to it (up, down, left or right) contains a pit: Bij ↔ (P(i−1)j ∨ P(i+1)j ∨

Pi(j−1) ∨ Pi(j+1)), omitting neighbours outside the grid. In the same way, Sij ↔ (W(i−1)j ∨ W(i+1)j ∨ Wi(j−1) ∨ Wi(j+1)).

For a 2 × 2 example, the percept rules are:

B11 ↔ (P12 ∨ P21)

B12 ↔ (P11 ∨ P22)

B21 ↔ (P11 ∨ P22)

B22 ↔ (P12 ∨ P21)

S11 ↔ (W12 ∨ W21), with equivalent neighbour rules for the other squares.

We can take the starting square (1,1) as safe. This gives us the facts ¬P11 and ¬W11 are facts. If the agent observes ¬B11 and ¬S11, then neither adjacent square (1,2) nor (2,1) contains a pit or Wumpus. So, based on these observations, both neighbouring squares can be treated as safe in this example.

To answer a query about a square, we check whether the query is true in every possible model that satisfies the knowledge base. In symbols, KB ⊨ α if and only if there is no model in which KB is true and α is false.

Student A – Design and Mathematical Analysis

Question: Count proposition symbols and sentences as a function of n, derive 2^m models, identify relevant symbols, and explain soundness and completeness.

Answer:

There are n^2 squares. With four basic proposition symbols (P, W, B and S) per square, the total number of proposition symbols is $m = 4n^2$. If Safe is stored as an additional independent symbol, the count becomes $5n^2$; in this design Safe is derived and is not counted as an independent symbol.

Grid size n	Squares n^2	Symbols $m = 4n^2$	All truth assignments 2^m
2	4	16	65,536
3	9	36	68,719,476,736
4	16	64	18,446,744,073,709,551,616

The exact sentence count can change depending on how we expand each biconditional and handle squares along the edges of the grid. A compact rule set contains one breeze relation and one stench relation per square, plus the safety definition and initial facts. This means the number of rule groups grows roughly with n^2, but the number of possible truth assignments increases much faster, as $2^{(4n^2)}$.

For a query about square (2,2), only the pit and Wumpus symbols of that square and its immediate neighbours, together with the percept symbols connected to those neighbours, are needed. Symbols in distant, disconnected regions can be excluded if no rule links them to the query. Focusing only on these relevant symbols can make the search much smaller.

Soundness means that if the checker reports KB ⊨ α, then α is true in every model of KB. Completeness means that if α is true in every model of KB, the checker will report entailment. Exhaustive truth-table model checking is sound and complete for finite propositional knowledge bases, provided all assignments are evaluated correctly.

Student B – Experimental Validation

Question: Record models enumerated, KB evaluations and time for n = 2, 3, 4, use relevant-symbol restriction, and test three query types.

Answer:

The table below gives the number of possible assignments for each grid size. Full enumeration is practical for n = 2, but becomes impractical at n = 3 and clearly infeasible at n = 4 on an ordinary student laptop. A relevant-symbol query can be evaluated using only the local variables connected to that query.

n	Full symbols	Full assignments	What this means in practice
2	16	65,536	Can be enumerated in a short experiment.
3	36	68.7 billion	Full enumeration is generally impractical.
4	64	1.84 × 10^19	Infeasible by direct enumeration.

For reproducibility, record the actual elapsed time from the implementation rather than inserting an estimated time. A suitable measurement record is: n, number of variables, assignments visited, KB evaluations, query, result and elapsed seconds.

Query type	Example KB information	Query	Expected result
Entailed	¬P11, ¬W11, ¬B11, ¬S11 and neighbour rules	Safe12	True / entailed in the simplified 2×2 example

Negation entailed	P22 is a fact	¬P22	False; its negation is entailed
Neither entailed	No fact or rule determines P22	P22	Undetermined; neither P22 nor ¬P22 follows

A query is entailed when it is true in all KB models. Its negation is entailed when the query is false in all KB models. If some models make it true and other models make it false, neither side is entailed.

Part B – Reasoning Patterns

Question: Derive the safety of at least two squares using Modus Ponens, And-Elimination and Unit Resolution.

Answer:

Example derivation using a 2 × 2 world:

Given ¬B11 and B11 ↔ (P12 ∨ P21), the breeze rule gives ¬(P12 ∨ P21).

By De Morgan's law, ¬P12 ∧ ¬P21.

By And-Elimination, derive ¬P12 and separately derive ¬P21.

Given ¬S11 and S11 ↔ (W12 ∨ W21), derive ¬W12 and ¬W21.

Using Safeij ↔ (¬Pij ∧ ¬Wij), apply Modus Ponens to derive Safe12 and Safe21.

The two safe-square conclusions are Safe12 and Safe21. The number of inference steps should be counted from the exact implementation; in the above hand derivation, the key derived literals are ¬P12, ¬P21, ¬W12, ¬W21, Safe12 and Safe21.

Student A – Experimental Investigation

Question: Implement the inference-rule derivation, record inference steps and compare with model checking.

Answer:

A rule-based system keeps a list of known facts and applies a rule whenever all of its conditions are met. For the example above, the engine derives the four negative hazard literals and then the two safety conclusions. Here, the work mainly depends on how many rules can be applied. Model checking, on the other hand, may need to examine as many as 2^m assignments. For a small KB, both are fast; as unrelated symbols are added, the rule-based derivation can avoid exploring assignments that do not contribute to the query.

facts = {"not_B11", "not_S11"}

```
rules = [
({"not_B11"}, "not_P12"),
({"not_B11"}, "not_P21"),
({"not_S11"}, "not_W12"),
({"not_S11"}, "not_W21"),
({"not_P12", "not_W12"}, "Safe12"),
({"not_P21", "not_W21"}, "Safe21"),
]
steps = 0
changed = True
while changed:
changed = False
for premises, conclusion in rules:
if premises.issubset(facts) and conclusion not in facts:
facts.add(conclusion)
steps += 1
changed = True
print("Derived:", sorted(facts))
print("Inference steps:", steps)
```

Note: This compact demonstration represents the already-expanded logical consequences of the percept rules. A full submission implementation should also parse the original biconditionals and record the model-checker metrics.

Student B – Soundness Reasoning

Question: Prove the soundness of each rule using truth tables and explain why not-derived is not false.

Answer:

Modus Ponens: from P and (P → Q), infer Q. The only row in which both premises are true is the row P = True and Q = True; therefore the conclusion is true whenever both premises are true.

P	Q	P → Q	Both premises P and (P→Q)	Q

T	T	T	T	T
T	F	F	F	F
F	T	T	F	T
F	F	T	F	F

And-Elimination: from P ∧ Q, infer P; and from P ∧ Q, infer Q. Whenever the conjunction is true, both components are true.

P	Q	P ∧ Q	Conclusion P valid?	Conclusion Q valid?
T	T	T	Yes	Yes
T	F	F	Not applicable	Not applicable
F	T	F	Not applicable	Not applicable
F	F	F	Not applicable	Not applicable

Unit Resolution: from (P ∨ Q) and ¬P, infer Q. If ¬P is true, P is false. For the disjunction to remain true, Q must be true. Thus the rule is sound.

P	Q	P ∨ Q	¬P	Q
T	T	T	F	T
T	F	T	F	F
F	T	T	T	T
F	F	F	T	F

If the rule engine does not derive a sentence, that does not automatically mean the sentence is false. The engine may be incomplete, may not have the necessary rule, or may have stopped before reaching the relevant rule. We can conclude that it is false only if we derive its negation or find a valid model in which it is false.

TASK 2 – The Quality of Reasoning Method: Model Checking versus Resolution

Question: Implement truth-table model checking and propositional resolution for a smart-building access-control policy containing at least 10 rules, including implication, biconditional and disjunction.

Answer:

Let V = valid badge, T = within permitted time, E = employee, G = registered guest, H = host approval, M = security manager approval, X = emergency override, A = access granted, D = door unlocked, and L = access logged.

Rule	Policy rule

R1	E → V
R2	G → V
R3	V → (A ∨ X)
R4	T → A
R5	G → H
R6	H → A
R7	X → M
R8	M → A
R9	A ↔ D
R10	A → L
R11	¬V → ¬A
R12	(E ∨ G) → L

The policy includes implication (→), biconditional (↔) and disjunction (∨). These rules are used only as an example for the assignment; they should not be treated as an actual building security policy. The knowledge base can be tested with different fact sets: a valid employee during permitted time; a guest without host approval; an emergency override without manager approval; and a person with an invalid badge.

Student A – Resolution Investigation

Question: Convert three sentences to CNF by hand, construct a resolution proof for one entailed query, measure clause statistics for chain lengths 10, 20, 40 and 80, and test a non-entailed query.

Answer:

CNF conversions:

V → (A ∨ X) ≡ ¬V ∨ A ∨ X.

A ↔ D ≡ (¬A ∨ D) ∧ (¬D ∨ A).

(E ∨ G) → L ≡ ¬(E ∨ G) ∨ L ≡ (¬E ∧ ¬G) ∨ L ≡ (¬E ∨ L) ∧ (¬G ∨ L).

Resolution proof: Given T and rule T → A, the CNF clauses are {T} and {¬T, A}. Resolving them on T gives {A}. Therefore KB ⊨ A.

For a non-entailed query such as A when the KB contains only ¬V and no emergency or independent access fact, the prover cannot derive A. The resolution process eventually stops when it cannot produce any new useful clause. If the process does not produce the empty clause,

we have not proved the query. For a finite propositional clause set, this process terminates because only finitely many distinct clauses can be formed.

Implication-chain length	Initial chain clauses	What we expect to see
10	Approximately 10–11	Small clause set; quick saturation.
20	Approximately 20–21	More possible resolvent pairs.
40	Approximately 40–41	Pair comparisons grow rapidly without indexing.
80	Approximately 80–81	Naive resolution can spend substantial time on irrelevant pairs.

Record actual clauses generated, attempted resolvents, discarded tautologies, removed subsumed clauses and elapsed time from the running prover. The table above gives the chain setup, not measured runtime results.

Student B – Model-Checking Comparative Analysis

Question: Record models and time, identify a family where model checking is faster and one where it is slower, and explain why.

Answer:

Model checking enumerates all assignments to the symbols in the KB. With m symbols, it may inspect 2^m assignments. It is attractive when m is small, the KB is compact, and the query can be evaluated directly. Resolution is attractive when the clauses are sparse and a short proof can be found without exploring most assignments.

KB family	Method likely to work better	Reason
Small KB with 6–10 symbols	Model checking	At most 1,024 assignments for 10 symbols; simple enumeration has low overhead.
Long sparse implication chain	Resolution	A query may follow a short chain of relevant clauses

		without enumerating every truth assignment.
Dense clauses with many complementary pairs	Depends on implementation	Resolution may generate many resolvents; model checking may be competitive for a small variable set.

For the required measurements, use the same KB and query for both methods. Report model assignments visited, clauses generated, resolvent attempts and elapsed time. For a fair comparison, both methods should be run on the same computer, using the same language and stopping conditions.

Joint Design Question

Question: What information is needed before choosing between model checking and resolution?

Answer:

Before choosing a method, I would look at how many symbols and clauses the knowledge base contains, how closely connected those clauses are, how often queries are asked, and whether the knowledge base changes often. I would also check whether we need a short proof or an explanation for each result. I would also measure the average number of models visited and resolution operations on representative KB families. For a small fixed vocabulary, model checking is simple and predictable. For sparse rule-based KBs with useful unit clauses, resolution may be more efficient. The choice should be based on measurements rather than KB size alone.

TASK 3 – Breaking the Expressiveness Barrier: Knowledge Engineering in First-Order Logic

Question: Build propositional and FOL encodings of university exam-eligibility regulations using the knowledge-engineering process.

Answer:

Task: decide whether a student is eligible to write a course examination. Knowledge sources: attendance rules, prerequisite completion, fee payment, disciplinary status and approved leave. Vocabulary: Student(x), Course(c), Attendance(x,c,a), RequiredAttendance(c,r),

ClearedPrerequisite(x,c) , FeesPaid(x,c), DisciplinaryHold(x), ApprovedLeave(x,c), Eligible(x,c).

In propositional logic, we would create separate symbols and rules for each student and course combination. With FOL, we can write the general rules once and store individual students and courses as facts. Example FOL rule: ∀x∀c[(Student(x) ∧ Course(c) ∧ AttendanceOK(x,c) ∧ PrerequisiteOK(x,c) ∧ FeesPaid(x,c) ∧ ¬DisciplinaryHold(x) ∧ (ApprovedLeave(x,c) ∨ AttendanceThresholdMet(x,c))) → Eligible(x,c)].

Student A – Design Derivation

Question: Define FOL vocabulary and at least 12 quantified axioms, including an existential statement, and derive growth of the two encodings.

Answer:

Predicates: Student(x), Course(c), AttendancePercent(x,c,a), AttendanceOK(x,c), PrerequisiteCleared(x,c), FeesPaid(x,c), DisciplinaryHold(x), ApprovedLeave(x,c), AttendanceRequirement(c,r), MeetsAttendance(x,c), Eligible(x,c), Ineligible(x,c), HasExam(x,c), and AdvisorAssigned(x).

Axioms (illustrative regulations):

A1. ∀x∀c[(Student(x) ∧ Course(c) ∧ AttendanceOK(x,c) ∧ PrerequisiteCleared(x,c) ∧ FeesPaid(x,c) ∧ ¬DisciplinaryHold(x)) → Eligible(x,c)].

A2. ∀x∀c[(Student(x) ∧ Course(c) ∧ ApprovedLeave(x,c) ∧ PrerequisiteCleared(x,c) ∧ FeesPaid(x,c) ∧ ¬DisciplinaryHold(x)) → Eligible(x,c)].

A3. ∀x∀c[Eligible(x,c) → Student(x)].

A4. ∀x∀c[Eligible(x,c) → Course(c)].

A5. ∀x∀c[(Student(x) ∧ Course(c) ∧ DisciplinaryHold(x)) → Ineligible(x,c)].

A6. ∀x∀c[(Student(x) ∧ Course(c) ∧ ¬FeesPaid(x,c)) → Ineligible(x,c)].

A7. ∀x∀c[(Student(x) ∧ Course(c) ∧ ¬PrerequisiteCleared(x,c)) → Ineligible(x,c)].

A8. ∀x∀c∀a∀r[(AttendancePercent(x,c,a) ∧ AttendanceRequirement(c,r) ∧ a ≥ r) → AttendanceOK(x,c)].

A9. ∀x∀c[AttendanceOK(x,c) → MeetsAttendance(x,c)].

A10. ∀x∀c[ApprovedLeave(x,c) → AttendanceException(x,c)].

A11. ∀x∀c[(Eligible(x,c) ∧ HasExam(x,c)) → MayWriteExam(x,c)].

A12. ∀x[AdvisorAssigned(x) → Student(x)].

A13. ∃x[Student(x) ∧ AdvisorAssigned(x)].

The existential axiom states that at least one student has an assigned advisor. It introduces an existence claim rather than naming a specific student.

For s students and c courses, a propositional design may need O(sc) instance facts and rule groups because each student-course pair has its own symbols. The number of general FOL rules stays almost the same as the database grows, although more facts are needed for additional students and courses. FOL therefore reduces repeated rule descriptions, although it requires unification and variable substitution during inference.

Student B – Crossover Investigation

Question: Design 15 competency queries (5 succeed, 5 fail, 5 return bindings), document three defects, and compare size/time as s and c grow.

Answer:

Example facts for testing: Student(Anu), Student(Bala), Course(BME101), Course(AI101), AttendancePercent(Anu,BME101,90), AttendanceRequirement(BME101,75), PrerequisiteCleared(Anu,BME101), FeesPaid(Anu,BME101), ¬DisciplinaryHold(Anu), and HasExam(Anu,BME101). Additional facts can be added for failing cases.

Category	Five example queries	Expected result
Success	Eligible(Anu,BME101); AttendanceOK(Anu,BME101); MeetsAttendance(Anu,BME101); Student(Anu); Course(BME101)	True
Failure	Eligible(Bala,AI101); FeesPaid(Bala,AI101); PrerequisiteCleared(Bala,AI101); ApprovedLeave(Bala,AI101); MayWriteExam(Bala,AI101)	False or not provable from the stated facts
Bindings	Eligible(x,BME101); AttendancePercent(Anu,c,a); AttendanceRequirement(c,r); Student(x); HasExam(x,c)	Return substitutions such as x=Anu, c=BME101, a=90, r=75 where supported

Three example KB defects and fixes:

Defect 1: The attendance rule used a > comparison, excluding students exactly at the required percentage. Fix: use a ≥ r comparison.

Defect 2: Approved leave was recorded but not connected to the eligibility rule. Fix: add the approved-leave eligibility alternative.

Defect 3: A disciplinary hold was missing from the eligibility condition. Fix: explicitly require ¬DisciplinaryHold(x).

Illustrative growth experiment plan:

Students (s)	Courses (c)	Pairs (s×c)	Propositional instance groups	FOL general axioms
10	5	50	Proportional to 50	12–13
100	20	2,000	Proportional to 2,000	12–13
500	50	25,000	Proportional to 25,000	12–13

Measure file size or stored sentence count and inference time using identical queries. The FOL rule count remains constant, but inference time can rise because the engine must unify variables with a larger set of facts. The point at which one method becomes faster depends on the implementation. We should find it by running the tests rather than guessing a value for the report.

Joint Conclusion

Question: Why does greater expressiveness not automatically give better reasoning performance for every KB size?

Answer:

FOL is more expressive because a single rule with variables can apply to many students and courses. However, the system still has to match those variables with facts, apply substitutions and check the possible matches. For a very small domain, a propositional KB may be faster because all relevant instances are already explicit. As the domain grows, FOL can save storage and rule duplication. Therefore, expressiveness improves representation and scalability of knowledge description, but does not guarantee lower inference time.

TASK 4 – The Reality of Unification and Lifted Inference

Question: Implement UNIFY, lifted Modus Ponens and propositionalisation; derive five most general unifiers (MGUs), including occurs-check and standardising-apart cases.

Answer:

A substitution maps variables to terms. A most general unifier (MGU) is a substitution that makes two expressions match while keeping the substitution as general as possible. The occurs check makes sure that a variable is not replaced with a term that already contains that variable.

Pair	MGU / result	Explanation
Knows(x, Ravi) and Knows(Anu, y)	{x/Anu, y/Ravi}	Both argument positions are unified.
Parent(x, y) and Parent(Anu, Bala)	{x/Anu, y/Bala}	Variables are replaced by matching constants.
Treat(x, x) and Treat(Anu, Anu)	{x/Anu}	Repeated variable must map consistently.
P(x) and P(f(x))	Failure with occurs check	x cannot be replaced by f(x), because x occurs inside f(x).
R(x, y) and R(y, x) after standardising apart	{x1/y2, y1/x2} (one valid renaming-based MGU)	Rename variables in one clause first so unrelated clause variables are not accidentally identified.

Before unifying two clauses, we standardise them apart by giving their variables different names. This keeps the variables in the two clauses independent. For example, rename Parent(x,y) from one rule to Parent(x1,y1) and the other clause variables to x2,y2. This prevents accidental variable capture.

Student B – Logical Design and Correctness

Question: Explain why lifted inference is sound and how it avoids generating all ground instances.

Answer:

Lifted inference works with variables directly. It uses unification to find the substitutions needed to answer a query. It is sound because the unifier makes the premises and conclusion consistent under a valid substitution, and the underlying inference rule preserves truth. This means we do not have to create every possible ground instance before starting the inference. Only substitutions relevant to the current inference are produced, reducing unnecessary work when the domain is large.

Student A – Experimental and Engineering Analysis

Question: Measure ground clauses for k = 3 to 30 and increasing predicate arity; introduce a function symbol and compare lifted and propositional inference.

Answer:

For a predicate of arity r over k constants, the number of possible ground atoms is k^r. If all tuples are allowed, the growth is:

Constants k	Unary (r=1)	Binary (r=2)	Ternary (r=3)
3	3	9	27
5	5	25	125
10	10	100	1,000
20	20	400	8,000
30	30	900	27,000

With a function symbol f of arity 1 and a term-depth bound d, the number of terms can grow rapidly. With k constants and one unary function, the terms up to depth d are approximately k(d+1) if only the chain f(c), f(f(c)), etc. is generated. With multiple function symbols or higher arity functions, the number can grow exponentially with depth. If we do not set a limit on term depth, the Herbrand universe can become infinite.

Lifted inference can avoid creating all k^r ground atoms if only a small number of substitutions match the query. Propositionalisation may be preferable when the domain is tiny and finite, all ground instances are reused many times, or a simple SAT/model-checking engine is already available. Actual inference steps and runtime must be measured on the same query and same KB.

Joint Engineering Decision

Question: When would propositionalisation be selected despite lifted inference?

Answer:

I would propositionalise when the domain is small, fixed and finite; the number of ground instances is manageable; and the same ground clauses are queried repeatedly. I would prefer lifted inference when the domain is larger, many possible groundings are irrelevant, or the KB contains reusable general rules. We should make this choice after comparing the number of ground clauses, successful unifications, inference steps and actual running time.

TASK 5 – Upgrading Resolution from Propositional to First-Order Logic

Question: Convert the four English statements to FOL and CNF, implement first-order resolution using the Task 4 unifier, and prove that an authorised employee exists.

Answer:

Predicates: Employee(x), Handled(x,d), Authorised(x), PatientData(d), Restricted(d), Audited(d), and Contains(R1, patient_data).

FOL translations:

$\forall x \forall d[(Employee(x) \land Handled(x,d) \land Restricted(d)) \rightarrow Authorised(x)]$.

$\forall d[PatientData(d) \rightarrow Restricted(d)]$.

$\exists x[Employee(x) \land \forall d(Audited(d) \rightarrow Handled(x,d))]$.

$PatientData(R1) \land Audited(R1)$.

CNF conversion:

Statement 1: $\neg Employee(x) \lor \neg Handled(x,d) \lor \neg Restricted(d) \lor Authorised(x)$.

Statement 2: $\neg PatientData(d) \lor Restricted(d)$.

Statement 3: Skolemise the existential employee using a Skolem constant e: Employee(e) and $\neg Audited(d) \lor Handled(e,d)$.

Statement 4: PatientData(R1) and Audited(R1).

Here, the Skolem constant e stands for an employee whose existence is stated in the third sentence. It is not the name of an actual employee; it is simply a new constant used in the logic.

Student A – Conversion and Proof Design

Question: Show conversion steps, justify Skolemisation, draw the proof tree and select a resolution strategy.

Answer:

Resolution proof:

From PatientData(R1) and $\neg PatientData(R1) \lor Restricted(R1)$, resolve to obtain Restricted(R1).

From Audited(R1) and $\neg Audited(R1) \lor Handled(e,R1)$, resolve to obtain Handled(e,R1).

We already have Employee(e).

Instantiate the first universal clause with x=e and d=R1: $\neg Employee(e) \lor \neg Handled(e,R1) \lor \neg Restricted(R1) \lor Authorised(e)$.

Resolve successively with Employee(e), Handled(e,R1) and Restricted(R1) to obtain Authorised(e).

So we can conclude that at least one authorised employee exists, and e is the witness for that conclusion. A set-of-support strategy is suitable because the negated query is placed in the support set and resolution focuses on clauses connected to the query.

Student B – Comparative Evidence

Question: Record clauses, resolution steps, proof length and unifications with/without strategy; test a non-entailed query under a step cap; compare with propositional resolution.

Answer:

For the proof above, the initial CNF contains six clause groups if the two facts in statement 4 are counted separately and the two Skolemised clauses are counted separately. The proof requires three principal intermediate derivations: Restricted(R1), Handled(e,R1), and Authorised(e). Exact unification counts depend on whether the implementation counts constant matches and variable substitutions separately.

A non-entailed query can be "Authorised(Anu)" when no fact identifies e with Anu and no rule establishes Anu's authorisation. If the step limit is reached, the prover should say "not proved within the limit" instead of saying that the query is false. First-order resolution can search an unbounded space when function symbols are present, so failure to find a proof within a finite cap does not establish non-entailment.

On an equivalent small finite domain, propositional resolution can ground the employee and document constants explicitly. It may be simpler and faster for a tiny domain, while FOL resolution avoids manually creating repeated ground clauses as the domain expands.

Cross-Understanding Requirement

Question: Explain why Skolemisation preserves satisfiability but not logical equivalence, and what the experiment shows about first-order resolution.

Answer:

During Skolemisation, an existential variable is replaced with a new Skolem constant or function. The new formula is equisatisfiable with the original one: the original formula has a model if and only if the Skolemised formula has a corresponding model in an expanded vocabulary. However, the formulas are not logically equivalent in the original vocabulary because the Skolem symbol is new and the transformed formula makes a particular witness explicit.

The experiment shows that first-order resolution has additional cost from unification, substitutions and potentially many ground terms. The non-entailed query demonstrates that a

prover may not terminate on every first-order input, especially with unrestricted function symbols. A step cap is useful operationally, but a capped search result must be reported as inconclusive.

TASK 6 – Advanced Inference Mini-Investigations

Question: Implement forward chaining and backward chaining over an organisation hierarchy with about 10 definite-clause rules and 100, 1,000 and 10,000 facts.

Answer:

Example predicates: ReportsTo(employee, manager), Manager(x), Approves(x, request), CanApprove(x, request), and SeniorManager(x). Example rules:

ReportsTo(x,y) ∧ Manager(y) → Manager(x).

Manager(x) ∧ Approves(x,r) → CanApprove(x,r).

ReportsTo(x,y) ∧ CanApprove(y,r) → CanApprove(x,r).

SeniorManager(x) → Manager(x).

Manager(x) ∧ ReportsTo(y,x) → Supervises(x,y).

Supervises(x,y) ∧ CanApprove(x,r) → CanApprove(y,r).

Manager(x) ∧ HasTraining(x) → TrainedManager(x).

TrainedManager(x) ∧ Approves(x,r) → ValidApproval(x,r).

ReportsTo(x,y) ∧ ReportsTo(y,z) → ReportsToIndirectly(x,z).

ValidApproval(x,r) → RequestApproved(r).

Student A – Forward Chaining Investigation

Question: Implement forward chaining to a fixed point and record inferred facts, rule firings, redundant derivations, iterations and irrelevant derived facts.

Answer:

Forward chaining is a data-driven method. It starts with the facts we already know, applies rules whose conditions are satisfied, adds the new conclusions and continues until nothing new can be inferred. A simple fixed-point engine is:

```
def forward_chain(facts, rules):
known = set(facts)
firings = 0
iterations = 0
changed = True
```

```
while changed:
changed = False
iterations += 1
for premises, conclusion in rules:
if premises.issubset(known):
firings += 1
if conclusion not in known:
known.add(conclusion)
changed = True
return known, firings, iterations
```

Forward chaining may derive many facts that are unrelated to a particular query because it attempts to reach a fixed point for the whole rule set. As the KB grows from 100 to 10,000 facts, the number of possible rule matches and redundant attempts can increase. Indexing facts by predicate and argument can reduce unnecessary scans.

Fact count	What to observe during testing	Likely issue
100	Small number of rule firings and iterations	Low overhead
1,000	More matches and derived facts	Repeated scans become visible
10,000	Potentially many irrelevant derived facts	Need indexing and duplicate suppression

Student B – Backward Chaining Investigation

Question: Implement backward chaining with substitutions, demonstrate a left-recursive non-terminating rule, apply a remedy and record goals/unifications/depth.

Answer:

Backward chaining works in the opposite direction. It starts with the question we want to answer and looks for rules that could prove it. The rule premises become subgoals. It can avoid processing large sections of the KB that are irrelevant to the requested goal.

A left-recursive rule such as Ancestor(x,z) :- Ancestor(x,y), Parent(y,z) can cause naive depth-first backward chaining to expand Ancestor repeatedly before reaching a base fact. One way to avoid this is to keep track of goals that have already appeared in the current search path.

Memoisation or a maximum depth limit can also help. A depth bound prevents infinite recursion but may miss valid proofs deeper than the limit.

```
def backward_chain(goal, facts, rules, path=None, depth=0, max_depth=50):
if path is None:


path = set()
if depth > max_depth or goal in path:
return False
if goal in facts:
return True
path = path | {goal}
for head, body in rules:
# In a full FOL implementation, unify head with goal
# and apply the resulting substitution to every body goal.
if head == goal:
if all(backward_chain(g, facts, rules, path, depth+1, max_depth)
for g in body):
return True
return False
```

The code is a propositional-shaped control-flow illustration; a complete FOL version must call the Task 4 UNIFY function and apply substitutions to the head and body. Record goals expanded, unifications, maximum recursion depth and repeated subgoals.

Verification Questions

Question: Why is inference direction central to performance, and why can backward chaining remain efficient with many irrelevant facts?

Answer:

The direction of inference decides whether the system works through the consequences of all known facts or focuses only on the rules needed for a particular query. Forward chaining is useful when many conclusions are needed or new facts arrive continuously. Backward chaining is useful when queries are specific and only a small part of the KB is relevant. With indexing by predicate and goal arguments, backward chaining can ignore unrelated facts and rules, even when the total fact count is large.

DAY-4 DESIGN CHALLENGE – TASK 7: Design a Knowledge-Based Reasoning Component from First Principles

Question: Design a hospital medication-safety advisor for patient records and prescribed drugs, scaling from 40 to 2,000 rules and 50,000 patients.

Answer:

The advisor takes details such as the patient's age, allergies, current medicines, diagnosed conditions and the newly prescribed medicine as input. It then gives a safety result along with an explanation that a person can understand. It is a decision-support system and must not replace the clinical judgement of a qualified healthcare professional.

Stage 1 – Independent Design Proposal

Student A – Initial proposal:

Use first-order logic because rules must apply to many patients and medicines. Vocabulary includes Patient(p), Drug(d), HasAllergy(p,a), ContainsIngredient(d,a), Takes(p,d), Contraindicated(d,c), HasCondition(p,c), and Unsafe(p,d). Use backward chaining for a specific patient-drug query. Explanations are generated by storing the rule and facts used in each proof.

Student B – Initial proposal:

Use a rule-based FOL knowledge base with a combination of forward chaining for patient-record updates and backward chaining for prescription checks. Use an indexed rule agenda and a trace of matched rules to produce explanations. Add an explicit "insufficient information" result when required patient data is missing.

These are draft independent proposals. The pair should retain the actual separately prepared versions and edit them only after discussion.

Stage 2 – Compare the Two Designs

Question: Identify agreement, differences, strengths and retained ideas.

Answer:

Aspect	Student A	Student B	Decision we agreed on
Representation	FOL	FOL	Use FOL for reusable patient/drug rules.
Inference	Backward chaining	Forward + backward chaining	Backward chaining for prescription queries; forward updates only when needed.
Scale strategy	General rules	Indexed agenda	Index by predicate, drug, ingredient and patient.
Explanation	Proof trace	Rule trace plus missing-data state	Store facts and rules supporting each result.

Stage 3 – Joint Final Design

Question: Give representation, vocabulary, inference method, indexing/agenda, explanation mechanism and complete pseudocode.

Answer:

Final representation: First-order definite clauses for deterministic rules, with explicit three-valued outcome handling: SAFE UNDER CHECKED RULES, UNSAFE, or INSUFFICIENT DATA. The system should not treat missing information as evidence of safety.

Vocabulary: Patient(p), Drug(d), Age(p,a), HasAllergy(p,i), Ingredient(d,i), Takes(p,d), Interaction(d1,d2), HasCondition(p,c), ContraindicatedFor(d,c), Dose(p,d,q), MaxDose(d,qmax), and Unsafe(p,d,reason).

Example rules:

HasAllergy(p,i) ∧ Ingredient(d,i) → Unsafe(p,d,allergy_match).

Takes(p,d1) ∧ Interaction(d,d1) → Unsafe(p,d,drug_interaction).

HasCondition(p,c) ∧ ContraindicatedFor(d,c) → Unsafe(p,d,condition_conflict).

Dose(p,d,q) ∧ MaxDose(d,qmax) ∧ q > qmax → Unsafe(p,d,dose_limit).

If all required checks have complete data and no unsafe rule fires, return SAFE UNDER CHECKED RULES.

Indexing strategy: index allergy rules by ingredient, interaction rules by drug pair, contraindication rules by condition and drug, and dose rules by drug. Use an agenda so only rules affected by a changed patient fact are reconsidered.

Explanation mechanism: every conclusion stores the rule ID, matched patient facts, matched drug facts and substitution. The explanation generator converts this proof trace into a sentence such as “Flagged because the patient record lists an allergy to ingredient X, and the prescribed medicine contains ingredient X.”

Pseudocode:

```
function checkPrescription(patient, prescribedDrug):
required = [allergy_data, current_medicines, conditions,
age, dose_information]
missing = findMissingRequiredData(patient, prescribedDrug, required)
if missing is not empty:
return INSUFFICIENT_DATA with explanation(missing)
candidateRules = retrieveIndexedRules(prescribedDrug, patient)
agenda = candidateRules
fired = set()
explanations = []
while agenda is not empty:
rule = agenda.pop()
match = unify(rule.premises, patient, prescribedDrug)
if match succeeds and (rule.id, match) not in fired:
fired.add((rule.id, match))
conclusion = apply(rule, match)
if conclusion is Unsafe:
explanations.append(rule.id, match, conclusion.reason)
if explanations is not empty:
return UNSAFE with explanations
else:
return SAFE_UNDER_CHECKED_RULES with checkedRuleList
```

A qualified clinician must review the result before any final decision is made. The system should record the knowledge-base version and timestamp so that each explanation can be audited.

Stage 4 – Divide the Validation Responsibilities

Question: Student A validates soundness/completeness and complexity; Student B tests at least 30 cases at 40, 400 and 2,000 rules with metrics and a baseline.

Answer:

Student A – Theoretical validation: For a definite-clause rule base, forward chaining is sound under standard Horn-clause semantics. With fair rule application and a finite set of ground facts, it reaches the least fixed point and is complete for ground atomic queries. A naive implementation may repeatedly scan all rules and facts; indexing and semi-naive evaluation reduce repeated work. Restricting the KB to definite clauses makes inference more manageable, but cannot directly represent every form of uncertainty or arbitrary non-Horn knowledge.

Student B – Experimental validation: Use 30 test cases covering allergy matches, drug interactions, contraindications, dose limits, safe cases, missing records and conflicting facts. Repeat the same cases at 40, 400 and 2,000 rules. Compare the proposed indexed system against a baseline that scans all rules.

Test group	Cases	Expected outcome
Allergy match / no match	1–5	Unsafe for matching ingredient; no allergy flag otherwise
Drug interaction	6–10	Unsafe when a known interacting medicine is present
Condition contraindication	11–15	Unsafe when the patient has a contraindicated condition
Dose limit	16–20	Unsafe when prescribed dose exceeds the stored limit

Missing or incomplete data	21–25	Insufficient data; do not assume safe
Safe and boundary cases	26–30	Safe under checked rules only when all required checks pass

Record query latency (milliseconds), inference steps, peak memory and explanation length. Use the same hardware and dataset for the baseline and proposed system. The table below is the measurement template; fill it with actual results:

Rules	Method
40	Full-scan baseline
40	Indexed design
400	Full-scan baseline
400	Indexed design
2,000	Full-scan baseline
2,000	Indexed design

Do not fill this table with invented benchmark numbers. Run the same 30 cases against both versions and insert the measured values.

Stage 5 – Cross-Defence

Question: Student A explains experimental evidence; Student B explains theoretical complexity.

Answer:

Student A should explain whether indexing reduced the number of rules examined, whether latency increased as the KB grew, and whether the explanation trace remained understandable. Student B should explain that query cost depends on the number of relevant rules and matching

facts, not only the total number of rules. In a naive implementation, repeated scans can approach a large product of rule count and fact count; indexing reduces the candidate set but does not remove the cost of unification or large numbers of matching facts.

Stage 6 – Joint Evaluation

Question: Why is this problem suitable for a logical agent? What are the limits of the representation, and what should an AI designer check?

Answer:

Medication-safety checking is suitable for a logical agent because many decisions are based on explicit relationships and rules: allergies, ingredients, interactions, contraindications and dose limits. A logical agent can provide traceable explanations showing which facts and rules caused a warning.

One important limitation is that patient information may be incomplete or uncertain. Patient records may be missing, outdated or inconsistent. Clinical risk may depend on context, probability, laboratory values and specialist judgement that simple Boolean rules cannot fully represent. Before choosing a knowledge-based approach, the designer should check whether reliable rules exist, whether the data are complete and current, how conflicts are resolved, how explanations are audited, how the system is validated, and how clinicians remain responsible for final decisions.